

ObjektinisV2.0_dok

Generated by Doxygen 1.13.2

1 Sistemos parametrai	1
1.0.1 Tyrimas: rule of five ir i/o operatoriai	1
1.0.2 Instrukcija	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Class Documentation	9
5.1 studentai Class Reference	9
5.1.1 Member Function Documentation	10
5.1.1.1 displayInfo()	10
5.1.1.2 pavarde()	10
5.1.1.3 vardas()	10
5.2 Zmogus Class Reference	11
6 File Documentation	13
6.1 antrastes.h	13
6.2 strukt_antr.h	14
Index	17

Chapter 1

Sistemos parametrai

CPU	RAM	SSD
i5-1240P 1.70 GHz	16 GB	SSD 512 GB

1.0.1 Tyrimas: rule of five ir i/o operatoriai

Atliekamas testavimas tų pačių funkcijų kaip ir v1.2, tikrinami operatoriai ir konstruktoriai, gaunamas atitinkamas veikimas ir rezultatai

1.0.2 Instrukcija

programą galima paleisti dviem būdais

1. sukompiliuojant komandinėje eilutėje

į terminalą reikia įrašyti:

```
g++ kodas_vektoriai.cpp funkcijos.cpp -o prog  
./prog
```

2. kompiliuojant su optimizavimo flag'ais (pvz. su O1, pataisyti atitinkamai su O2 ir O3), *paskutinė eilutė .exe failo dydžio (KB) gavimas*

```
g++ -O1 kodas_vektoriai.cpp funkcijos.cpp -o prog_O1  
./prog_O1  
{0:N2} KB -f ((Get-Item .\prog_O1.exe).Length / 1024)
```

3. naudojant **cmake** failą

į terminalą reikia įrašyti (jei jau buvo sukurtas build aplankas):

```
rm -r build  
mkdir build  
cd build  
cmake ..  
cmake --build .  
cd debug  
.\Objektinis.exe
```


Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	11
studentai	9

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

studentai		9
Zmogus		11

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

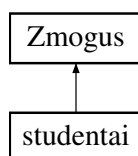
antrastes.h		13
strukt_antr.h		14

Chapter 5

Class Documentation

5.1 studentai Class Reference

Inheritance diagram for studentai:



Public Member Functions

- void **setVardas** (const string &vardas)
- void **setPavarde** (const string &pavarde)
- **studentai** (std::istream &is)
- **studentai** (const [studentai](#) &v)
- **studentai** ([studentai](#) &&v) noexcept
- [studentai](#) & **operator=** (const [studentai](#) &v)
- [studentai](#) & **operator=** ([studentai](#) &&v) noexcept
- void [displayInfo](#) () const override
- double **egzam** () const
- const vector< double > & **pazymiai** () const
- double **suma** () const
- double **vidurkis** () const
- int **mediana** () const
- double **gal_med** () const
- double **gal_vid** () const
- std::istream & **readStudent** (std::istream &)
- void **setEgzam** (double egzam)
- void **setPazymiai** (const vector< double > &pazymiai)
- void **setSuma** (double suma)
- void **setVidurkis** (double vidurkis)
- void **setMediana** (int mediana)
- void **setGalMed** (double gal_med)
- void **setGalVid** (double gal_vid)
- void **addPazymys** (double pazymys)
- virtual string [vardas](#) () const
- virtual string [pavarde](#) () const

Public Member Functions inherited from [Zmogus](#)

- **Zmogus** (string var="", string pav="")
- void **setVardas** (string var)
- void **setPavarde** (string pav)

Friends

- std::istream & **operator>>** (std::istream &is, [studentai](#) &v)
- std::ostream & **operator<<** (std::ostream &os, const [studentai](#) &v)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string **vardas_**
- string **pavarde_**

5.1.1 Member Function Documentation

5.1.1.1 displayInfo()

```
void studentai::displayInfo () const [inline], [override], [virtual]
```

Implements [Zmogus](#).

5.1.1.2 pavarde()

```
virtual string Zmogus::pavarde () const [inline], [virtual]
```

Reimplemented from [Zmogus](#).

5.1.1.3 vardas()

```
virtual string Zmogus::vardas () const [inline], [virtual]
```

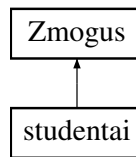
Reimplemented from [Zmogus](#).

The documentation for this class was generated from the following files:

- strukt_antr.h
- kodash_vektoriai.cpp

5.2 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** (string var="", string pav="")
- void **setVardas** (string var)
- void **setPavarde** (string pav)
- virtual string **vardas** () const
- virtual string **pavarde** () const
- virtual void **displayInfo** () const =0

Protected Attributes

- string **vardas_**
- string **pavarde_**

The documentation for this class was generated from the following file:

- `strukt_antr.h`

Chapter 6

File Documentation

6.1 antrastes.h

```
00001 #ifndef ANTRASTES_H
00002 #define ANTRASTES_H
00003
00004 #include "strukt_antr.h"
00005
00006 #include <iostream>
00007 #include <iomanip>
00008 #include <vector>
00009 #include <string>
00010 #include <algorithm> // kad veiktu sort
00011 #include <ctime> //rand
00012 #include <random>
00013 #include <fstream>
00014 #include <sstream>
00015 #include <chrono> //laiko skaiciavimui
00016 #include <ctime>
00017 #include <numeric> //accumulate
00018 #include <iterator> //istream_iterator
00019
00020 using std::vector;
00021 using std::string;
00022
00023 void vartotojo_pasirinkimas(int &nr_meniu, int a, int b);
00024 bool ar_beda(int x, int pr, int pb);
00025 bool ar_beda_bendras(int x, int pr, int pb);
00026 int rand_pazymys();
00027 void failo_generavimas(string failo_pavadinimas, int nr_failo_dydis, int paz_kiekis);
00028 int mediana_skaiciavimas(vector <int> &pazymiai, studentai &temp);
00029 string vardo_generavimas();
00030 string pavardes_generavimas();
00031
00032 void spausdinimas(vector<studentai> grupe);
00033 void spausdinimas_faile(vector <studentai> grupe, const string& outputo_pavadinimas);
00034
00035 void rikiavimas(int nr_rikiavimas, vector<studentai> &grupe);
00036 void spausdinimo_parinkimas(vector<studentai> grupe, int nr_spausdinimas, int nr_rikiavimas);
00037
00038
00039 int sumos_skaiciavimas(vector <int> &pazymiai, studentai &temp);
00040 double vidurkio_skaiciavimas(vector <int> &pazymiai, studentai &temp);
00041 double galutinis_vid_sk(studentai &temp, double &vidurkis);
00042 double galutinis_med_sk(studentai &temp, int &mediana);
00043
00044 /*
00045 template <typename Container>
00046 void spausdinimas_faile(const Container& grupe, const std::string& outputo_pavadinimas);
00047 template <typename Container>
00048 void nuskaitymas(const std::string& failo_pavadinimas, Container& grupe, int &n);
00049 template <typename Container>
00050 void skaiciavimas(Container& grupe, int n);
00051 */
00052
00053 void nuskaitymas(const string& failo_pavadinimas, vector<studentai>& grupe, int &n);
00054 void skaiciavimas(vector<studentai>& grupe, int n);
00055
00056 #endif // ANTRASTES_H
```

6.2 strukt_antr.h

```

00001 #ifndef strukt_antr_h
00002 #define strukt_antr_h
00003
00004 #include <iostream>
00005 #include <vector>
00006 #include <string>
00007
00008 using std::vector;
00009 using std::string;
00010
00011 class Zmogus
00012 {
00013     protected:
00014         string vardas_;
00015         string pavarde_;
00016     public:
00017         Zmogus(string var="", string pav="") : vardas_{var}, pavarde_{pav} {std::cout << "Zmogaus
konstruktorius \n";} // default konstruktorius
00018         void setVardas(string var) { vardas_ = var; }
00019         void setPavarde(string pav) { pavarde_ = pav; }
00020         virtual string vardas() const { return vardas_; }
00021         virtual string pavarde() const { return pavarde_; }
00022
00023         virtual void displayInfo() const = 0; // virtual = abstract class
00024         virtual ~Zmogus() {
00025             vardas_.clear();
00026             pavarde_.clear();
00027             //std::cout << "Destruktorius Zmogus įvykdytas" << std::endl;
00028         }
00029 };
00030 //isvestine klase
00031 class studentai: public Zmogus
00032 {
00033     private:
00034         double egzam_;
00035         vector <double> pazymiai_;
00036         double suma_=0;
00037         double vidurkis_=0;
00038         int mediana_=0;
00039         double gal_med_ = 0, gal_vid_=0;
00040     public:
00041         void setVardas(const string& vardas) {
00042             Zmogus::setVardas(vardas); // Optionally call the base class method
00043         }
00044
00045         void setPavarde(const string& pavarde) {
00046             Zmogus::setPavarde(pavarde); // Optionally call the base class method
00047         }
00048
00049         //konstruktoriai:
00050         studentai() : Zmogus(), egzam_(0) {} // default konstruktorius
00051         studentai(std::istream& is); //dis: konstruktorius, realiai inicializacijai
00052
00053         studentai(const studentai& v); //copy konstruktorius
00054         studentai(studentai&& v) noexcept; //move konstruktorius
00055         //studentai(std::initializer_list<double> il); // perduodamas double sarasas
00056         //kas cia?
00057         studentai& operator=(const studentai& v); // priskyrimo kopijavimo operatorius
00058         studentai& operator=(studentai&& v) noexcept; // priskyrimo move operatorius
00059
00060         using Zmogus::vardas;
00061         using Zmogus::pavarde;
00062
00063         void displayInfo() const override {
00064             std::cout << "Studentas: " << vardas_ << " " << pavarde_ << std::endl;
00065         }
00066         // this
00067
00068         //get'eriai
00069         //string vardas() const {return vardas_;}
00070         //string pavarde() const {return pavarde_;}
00071         double egzam() const { return egzam_; }
00072         const vector<double>& pazymiai() const { return pazymiai_; }
00073         double suma() const {return suma_;}
00074         double vidurkis() const;
00075         int mediana() const;
00076         double gal_med() const;
00077         double gal_vid() const;
00078
00079         std::istream& readStudent(std::istream&);
00080         //set'eriai
00081         void setEgzam(double egzam) {egzam_ = egzam;}
00082         void setPazymiai(const vector<double>& pazymiai) {pazymiai_ = pazymiai;}
00083

```

```

00084     void setSuma(double suma) {suma_ = suma;}
00085     void setVidurkis(double vidurkis) {vidurkis_ = vidurkis;}
00086     void setMediana(int mediana) {mediana_ = mediana;}
00087     void setGalMed(double gal_med) {gal_med_ = gal_med;}
00088     void setGalVid(double gal_vid) {gal_vid_ = gal_vid;}
00089
00090     void addPazymys(double pazymys) {pazymiai_.push_back(pazymys);}
00091
00092
00093     //perdengti operatoriai
00094     // i/o operatoriai friend tipo
00095     friend std::istream& operator>>(std::istream& is, studentai& v);
00096     friend std::ostream& operator<<(std::ostream& os, const studentai& v);
00097
00098
00099     //destruktorius:
00100     ~studentai() override {
00101         pazymiai_.clear();
00102         //std::cout << "Destruktorius išvestinės studentai įvykdytas" << std::endl;
00103     }
00104     //override - virtualus išvestinės destruktorius
00105 };
00106
00107
00108
00109
00110 /*
00111 class studentai
00112 {
00113     private:
00114         string vardas_;
00115         string pavarde_;
00116         double egzam_;
00117         vector<double> pazymiai_;
00118         double suma_=0;
00119         double vidurkis_=0;
00120         int mediana_=0;
00121         double gal_med_ = 0, gal_vid_=0;
00122     public:
00123         //konstruktoriai:
00124         studentai() : egzam_(0) {} // default konstruktorius
00125         studentai(std::istream& is); //dis: konstruktorius, realiai inicializacijai
00126
00127         studentai(const studentai& v); //copy konstruktorius
00128         studentai(studentai&& v) noexcept; //move konstruktorius
00129         studentai(std::initializer_list<double> il); // perduodamas double sarasas
00130         //kas cia?
00131         studentai& operator=(const studentai& v); // priskyrimo kopijavimo operatorius
00132         studentai& operator=(studentai&& v) noexcept; // priskyrimo move operatorius
00133
00134
00135         //get'eriai, inline:
00136         inline string vardas() const {return vardas_;}
00137         inline string pavarde() const {return pavarde_;}
00138         inline double egzam() const { return egzam_; }
00139         inline const vector<double>& pazymiai() const { return pazymiai_; }
00140         inline double suma() const {return suma_;}
00141         inline double vidurkis() const;
00142         inline int mediana() const;
00143         double gal_med() const;
00144         double gal_vid() const;
00145
00146         //set'eriai:
00147         std::istream& readStudent(std::istream&); //set'eriai
00148         void setVardas(string vardas) {vardas_ = vardas;}
00149         void setPavarde(string pavarde) {pavarde_ = pavarde;}
00150         void setEgzam(double egzam) {egzam_ = egzam;}
00151         void setPazymiai(const vector<double>& pazymiai) {pazymiai_ = pazymiai;}
00152
00153         void setSuma(double suma) {suma_ = suma;}
00154         void setVidurkis(double vidurkis) {vidurkis_ = vidurkis;}
00155         void setMediana(int mediana) {mediana_ = mediana;}
00156         void setGalMed(double gal_med) {gal_med_ = gal_med;}
00157         void setGalVid(double gal_vid) {gal_vid_ = gal_vid;}
00158
00159         void addPazymys(double pazymys) {pazymiai_.push_back(pazymys);}
00160
00161
00162         //perdengti operatoriai
00163         // i/o operatoriai friend tipo
00164         friend std::istream& operator>>(std::istream& is, studentai& v);
00165         friend std::ostream& operator<<(std::ostream& os, const studentai& v);
00166
00167
00168         //destruktorius:
00169         ~studentai() {
00170             pazymiai_.clear();

```

```
00171         vardas_.clear();
00172         pavarde_.clear();
00173         std::cout << "Destruktorius įvykdytas" << std::endl;
00174     }
00175 };
00176 */
00177
00178
00179 //bool compare(const studentai&, const studentai&);
00180 /*
00181 bool comparePagalPavarde(const studentai&, const studentai&);
00182 bool comparePagalEgza(const studentai&, const studentai&);
00183 */
00184
00185 /*
00186 struct studentai
00187 {
00188     string vardas="var"; //default reikšmes testavimui
00189     string pavarde="pav";
00190     vector<int> pazymiai; //nd tarpiniai rezultatai
00191     int egzam;
00192     double suma=0;
00193     double vidurkis;
00194     int mediana;
00195     double gal_vid, gal_med;
00196 };
00197 */
00198
00199 #endif
```

Index

displayInfo
 studentai, [10](#)

pavarde
 studentai, [10](#)

Sistemas parametrai, [1](#)
studentai, [9](#)
 displayInfo, [10](#)
 pavarde, [10](#)
 vardas, [10](#)

vardas
 studentai, [10](#)

Zmogus, [11](#)